

Guía de Estudio — Clase 13B: Módulos, Interfaces y Genéricos

Asignatura: Laboratorio de Programación y Lenguajes 2026 (IF009) **Institución:** Universidad Nacional de Tierra del Fuego — Instituto IDEI **Módulo:** X — Abstracción y Modularidad | **Semana:** 13 | **Clase:** 13B (1 de 1) **Tema (topic.yaml):** Módulos, Interfaces y Genéricos **Duración:** 120 minutos | **Lenguaje principal:** TypeScript **Contrastes:** Go, Rust, Swift, Kotlin **Bibliografía principal:** Sebesta 2019, Cap. 9 (Subprograms) y Cap. 10 (Implementing Subprograms) **Docente:** Matías Gel **Fecha de generación:** 2026-06-28

! Drift detectado

Importante — leé esto antes de empezar.

El tema se llama “**Módulos, Interfaces y Genéricos**” en el plan oficial (`topic.yaml` , Clase 13B), pero el contenido **realmente dictado en clase** —registrado en `clase_dada.txt` (607 líneas)— trata sobre “**Subprogramas: del contrato a la ejecución**”: contrato, perfil/protocolo, parámetros, modos de pasaje, mecanismos, callbacks, sobrecarga, dispatch, genéricos, activation records, dynamic/static link y async.

Existe un tema gemelo `13-subprogramas-parametros-sobrecarga` (Clase 13A) cuyo `clase_dada.txt` trata sobre TAD/módulos/interfaces/genéricos — lo inverso. Pareciera que los dos `clase_dada.txt` están **intercambiados** entre las carpetas 13.

Decisión operativa (Dr. Roberto, `diseño.md`): esta guía es **fiel al `clase_dada.txt`** (subprogramas), manteniendo el título oficial del tema por consistencia con el sistema. Si al estudiar ves que el título no coincide con el contenido, **no es un error tuyo:** es el drift documentado. Ver `diseño.md` (sección “! Drift detectado”) para el contexto completo.

En resumen: **lo que vas a estudiar acá es subprogramas**, aunque el plan lo llame “Módulos, Interfaces y Genéricos”.

1. Introducción al tema

Esta clase es el puente entre dos mundos que venimos construyendo a lo largo del cursado: **el contrato** (lo que el cliente de un subprograma ve y firma) y **el mecanismo de ejecución** (lo que pasa en memoria cuando ese contrato se invoca).

Hasta ahora trabajamos abstracción desde el lado del dato (TADs, tipos, polimorfismo) y desde el lado del control (estructuras, excepciones). Acá bajamos al **subprograma** como unidad de modularización procedural: la abstracción de acción más básica que ofrece casi todo lenguaje. La pregunta conductora no es “¿cómo se declara una función?” —eso ya lo sabés— sino “¿qué contrato

establece un subprograma con quien lo invoca, y cómo se materializa ese contrato cuando se ejecuta?”.

El recorrido es deliberado: del contrato visible (perfil, protocolo, modos, efectos) al mecanismo de ejecución (activation record, call/return, async, enlaces). Cada decisión de diseño que veamos en el contrato tiene una contraparte concreta en la ejecución. Por eso la clase cierra conectando seis preguntas en una sola síntesis.

 **Nota de Sofía:** si podés estudiar esta guía solo, sin abrir la clase, hicimos bien el trabajo. Si algo no se entiende, escribime la duda en el margen: cada sección tiene una pregunta de autoevaluación al final para que verifiques.

2. Objetivos de aprendizaje

Al terminar esta guía, vas a poder:

1. **Explicar** un subprograma como abstracción de acción que se razona por **contrato**, no por instrucciones.
2. **Distinguir** definición y llamada como roles distintos que forman un contrato.
3. **Diferenciar** procedimiento y función por intención, resultado y composición.
4. **Usar** perfil y protocolo para verificar llamadas sin leer el cuerpo.
5. **Identificar** los modos de pasaje (`in` , `out` , `inout`) como dirección del flujo de información.
6. **Relacionar** intención (consultar, modificar, consumir, producir) con **permiso mínimo**.
7. **Reconocer** efectos observables (mutación, falla, suspensión, cancelación) en el contrato moderno.
8. **Comparar** los cinco mecanismos de pasaje (valor, resultado, valor-resultado, referencia, nombre) por ventajas y riesgos.
9. **Explicar** pass-by-value en Go, aliasing mutable restringido en Rust e `inout` en Swift.
10. **Distinguir** pass-by-sharing de pass-by-reference: separar variable y objeto.
11. **Explicar** un callback como parte del contrato del llamador, incluyendo síncrono, suspendible y escapante.
12. **Comparar** sobrecarga, unión sellada, genérico/trait e interfaz dinámica como herramientas de variación.
13. **Diferenciar** dispatch estático (`impl Trait`) y dinámico (`dyn Trait`) en Rust.
14. **Explicar** los costos de la abstracción genérica: especialización, implementación compartida, preservación selectiva.
15. **Describir** los componentes de un activation record y la secuencia call/return.
16. **Explicar** `async` como extensión del modelo de ejecución con máquina de estados reanudable.
17. **Distinguir** dynamic link (¿quién me llamó?) y static link (¿dónde busco variables no locales?).
18. **Sintetizar** que las decisiones de diseño del subprograma están conectadas: del contrato visible al mecanismo de ejecución.

📖 **Cómo leer los objetivos:** los verbos no son decorativos. *Explicar* y *diferenciar* piden que digas la distinción con tus palabras; *comparar* pide una tabla mental; *sintetizar* pide que conectes todo. Si llegás al objetivo 18, cerraste el hilo conductor de la clase.

3. Conceptos previos necesarios

Esta clase **no re-explica** los temas anteriores; los usa. Antes de empezar, asegurate de tener fresco:

Tema previo	Qué necesitás recordar	Dónde está
09 — Variables, Binding y Ámbito	Ámbito estático vs. dinámico; reglas de visibilidad de nombres. El static link se apoya acá.	Tema 09
09.2 — Aliases, Closures, GC	Qué es una closure (captura de entorno) y por qué un callback escapante puede retener referencias. La clase dice <i>"la closure ya fue estudiada; aquí importa su impacto contractual"</i> .	Tema 09.2
10 — Tipos de Datos y Sistemas de Tipos	Verificación de tipos, compatibilidad. El perfil/protocolo es verificación de tipos aplicada a subprogramas.	Tema 10
10.2 — Tipos Compuestos y Polimorfismo	Polimorfismo paramétrico y ad hoc. La clase retoma estos nombres al hablar de genéricos y sobrecarga.	Tema 10.2
11 — Expresiones y Estructuras de Control	Secuencia, selección, iteración. El procedimiento "se encadena por secuencia"; la función "se compone dentro de expresiones".	Tema 11
12 — Manejo de Excepciones	Falla como efecto observable. La clase trata <code>Result<T,E></code> y excepciones documentadas como parte del contrato moderno.	Tema 12

Si alguno de estos temas está borroso, no hace falta que lo reestudies completo: con tener la idea central (ámbito, closure, verificación de tipos, polimorfismo, secuencia, excepción) alcanza para seguir esta guía.

4. Mapa de la clase

La clase se organiza en **9 bloques** que recorren el hilo *contrato* → *ejecución*:

Bloque	Filminas	Tema	Tiempo
A	F-01 a F-04	El subprograma como abstracción de acción	20 min
B	F-05 a F-08	Parámetros: modos, permisos y efectos	18 min
C	F-09 a F-11	Tres lenguajes, tres decisiones de pasaje	18 min
D	F-12 a F-13	Compartir objetos: copia vs. aliasing	8 min
E	F-14 a F-15	Callbacks: contrato, suspensión y escape	12 min
F	F-16 a F-18	Variación: sobrecarga, dispatch y trait	14 min
G	F-19	Abstracción genérica y sus costos	6 min
H	F-20 a F-23	Ejecución: activation records y async	19 min
I	F-24	Cierre: decisiones conectadas	5 min

Cada sección del desarrollo teórico referencia la filmina correspondiente con [F-XX] para que puedas cruzar con las filminas de la clase.

5. Desarrollo teórico

5.1 Bloque A — El subprograma como abstracción de acción [F-01 a F-04]

5.1.1 Un subprograma abstrae una acción [F-01]

Un **subprograma** es una abstracción de acción: permite razonar por **contrato**, no por instrucciones. Cuando llamás a `console.log` no lees el cuerpo — confiás en el contrato. Esa es la idea central.

Sus propiedades:

- Tiene un **punto de entrada único** y devuelve el control al llamador.
- Su encabezado establece qué datos recibe y qué resultado produce.
- El cuerpo queda **oculto** durante el uso: el cliente invoca una abstracción.
- Cada llamada crea una **activación distinta** del mismo código. El código es uno; las activaciones son muchas.

 **Diferencia clave:** *código* $\neq$ *activación*. El código de `distancia` se escribe una vez; cada llamada crea una activación nueva con sus propios parámetros y locales. Esta distinción es la base del activation record que veremos en el Bloque H.

Cita: Sebesta define los tres conceptos fundacionales: “A subprogram definition describes the interface to and the actions of the subprogram abstraction. A subprogram call is the explicit request that a specific subprogram be executed. A subprogram is said to be active if, after having been called, it has

begun execution but has not yet completed that execution.” [Sebesta 2019, §9.2 Fundamentals of Subprograms, p. 389]

Gabbrielli & Martini ubican al subprograma como unidad de modularización: *“The concept of procedure, or function, or subprogram, constitutes the fundamental unit of program modularisation. Communication between procedures is effected using return values, parameters and the nonlocal environment.”* [Gabbrielli & Martini 2023, Cap. 7, p. 136]

5.1.2 Definición y llamada forman un contrato [F-02]

La función `distancia` del `clase_dada.txt` :

```
function distancia(x1: number, y1: number,
                  x2: number, y2: number): number {
  return Math.hypot(x2 - x1, y2 - y1);
}
```

La tabla que tenés que saber de memoria es la que distingue los **roles** de cada elemento en definición vs. llamada:

Elemento	En la definición	En la llamada
Nombre	Identifica el servicio	Selecciona el servicio
Parámetros	Formales: variables locales del llamado	Reales: valores o expresiones aportadas
Perfil	Número, orden y tipos de parámetros	Debe ser compatible con los argumentos
Protocolo	Perfil más tipo de retorno	Determina el tipo de la expresión resultante
Cuerpo	Implementa el servicio	Permanece oculto al cliente

📖 **Trampa frecuente:** el **perfil NO incluye el tipo de retorno**. El perfil es solo parámetros (número, orden, tipos). El **protocolo** es perfil + tipo de retorno. Esta distinción vuelve a aparecer en sobrecarga: un subprograma sobrecargado debe tener un protocolo único [Sebesta 2019, §9.9, p. 389].

Cita: *“The parameters in the subprogram header are called formal parameters. They are sometimes thought of as dummy variables because they are not variables in the usual sense: In most cases, they are bound to storage only when the subprogram is called, and that binding is often through some other program mechanism.”* [Sebesta 2019, §9.2, p. 389]

5.1.3 Procedimiento vs. función [F-03]

Decisión	Procedimiento	Función
Intención principal	Cambiar el estado o producir un efecto	Calcular un valor
Resultado	Implícito en el estado modificado	Explícito mediante retorno
Composición	Se encadena por secuencia	Se compone dentro de expresiones
Contraste moderno	Kotlin usa <code>Unit</code> para efectos	Rust exige declarar el tipo retornado cuando no es <code>()</code>
Riesgo	Efectos difíciles de rastrear	Dependencia de entradas y entorno

No es “mejor” uno u otro: **expresan intenciones diferentes**. La pregunta es cuál intención querés comunicar. El retorno distingue cálculo de efecto.

Cita: “The term ‘procedure’ should denote a subprogram which does not directly return a value, while a function is a subprogram that returns one.” [Gabbrielli & Martini 2023, Cap. 7, p. 106]

5.1.4 Perfil y protocolo: verificar sin leer el cuerpo [F-04]

El **perfil** incluye cantidad, orden y tipos de los parámetros. El **protocolo** agrega el tipo de retorno. El chequeo estático rechaza llamadas incompatibles **antes de ejecutar**.

```
type Distancia = (  
  x1: number, y1: number,  
  x2: number, y2: number  
) => number;  
  
const euclidea: Distancia = (x1, y1, x2, y2) =>  
  Math.hypot(x2 - x1, y2 - y1);  
  
const resultado = euclidea(0, 0, 3, 4); // 5
```

El contrato permite validar sin leer el cuerpo: el cliente razona con información visible. Pero **el contrato reduce conocimiento necesario, no describe toda la semántica** — el tipo no captura efectos como mutación, falla o suspensión (lo veremos en F-07).

 **Anticipación:** el mismo concepto de protocolo **reaparece** en sobrecarga (F-16/F-17) y en funciones de orden superior (F-14). Si lo entendés acá, lo entendés en todos lados.

Cita: “An overloaded subprogram must have a unique protocol; that is, it must be different from the others in the number, order, or types of its parameters, and possibly in its return type.” [Sebesta 2019, §9.9, p. 389]

5.2 Bloque B — Parámetros: modos, permisos y efectos [F-05 a F-08]

5.2.1 La dirección del flujo: in, out, inout [F-05]

Antes del mecanismo, importa la **dirección del flujo** de información:

- **Modo in** : el llamado recibe información del llamador.
- **Modo out** : el llamado produce información para el llamador.
- **Modo inout** : la información circula en ambas direcciones.

La elección debería **minimizar acceso innecesario** a datos externos. Luego se elige un mecanismo que implemente ese modo. **Primero pensás la dirección; después el mecanismo. No al revés.**

 **Trampa frecuente:** `in` (modo) **no es lo mismo** que `pass-by-value` (mecanismo). `in` es la dirección del flujo; `pass-by-value` es un mecanismo que *implementa* `in`. También se puede implementar `in` con referencia inmutable (`read-only`).

Cita: “From a semantic viewpoint, there are in modes, out modes and inout modes.” [Gabbrielli & Martini 2023, Cap. 7, p. 136]. Sebesta desarrolla los modos en §9.5 Parameter-Passing Methods [Sebesta 2019, §9.5, p. 389].

5.2.2 Intención y permiso mínimo [F-06]

Los efectos exigen **permisos** sobre los datos. El contrato debe limitar qué puede hacer el subprograma:

Intención	Permiso mínimo	Evidencia moderna
Consultar	Lectura compartida	Referencia inmutable, <code>readonly</code>
Modificar	Acceso mutable exclusivo	<code>&mut</code> , <code>inout</code>
Consumir	Transferencia de ownership	Parámetro por valor no copiable
Producir	Retorno tipado	Valor, <code>Result</code> o promesa

El principio es **permiso mínimo, no máximo**: menos superficie de error. Si pedís más permiso del necesario, el contrato permite efectos que no necesitás y eso genera bugs difíciles de rastrear.

Cita: “Constant parameters clearly implement in-mode semantics.” [Sebesta 2019, §9.5, p. 389]. Y Gabbrielli: “When the formal parameter is not modified in the body of the function, we can imagine maintaining the semantics of passing by value, implementing it using call by reference. This is what constitutes the read-only parameter.” [Gabbrielli & Martini 2023, Cap. 7, p. 136]

5.2.3 Efectos observables en el contrato moderno [F-07]

La firma tipada **no siempre cuenta toda la historia**. El contrato moderno incluye efectos observables:

Efecto	Evidencia en el contrato	Consecuencia para el llamador
Retorno normal	Tipo de retorno	Composición directa
Mutación	<code>&mut</code> , <code>inout</code> , objeto mutable	Estado compartido observable
Falla	<code>Result<T, E></code> , excepción documentada	Flujo alternativo
Suspensión	<code>async</code> , <code>suspend</code>	Continuación diferida
Cancelación	Señal o contexto	Terminación cooperativa

Por eso Rust usa `Result` , Kotlin usa `suspend` y Swift usa `@escaping` . **El efecto es parte del contrato.** Una excepción no documentada es un efecto invisible: el llamador no puede manejarla.

Cita: Sebesta trata los side effects en el diseño de funciones en §9.8 [Sebesta 2019, §9.8, p. 389] y los modos in/out/inout en §9.5 [Sebesta 2019, §9.5, p. 389].

5.2.4 Mecanismos de pasaje: tradeoffs [F-08]

Cinco mecanismos clásicos, cada uno implementa un modo y tiene una ventaja y un riesgo:

Mecanismo	Implementa	Ventaja	Riesgo o costo
Valor	<code>in</code>	Aísla al llamador	Copiar objetos grandes
Resultado	<code>out</code>	Expresa salida	Colisión al copiar resultados
Valor-resultado	<code>inout</code>	Evita aliasing durante la llamada	Orden de copia al retornar
Referencia	<code>inout</code>	Evita copias grandes	Aliasing y efectos laterales
Nombre	<code>inout</code>	Reevalúa expresiones	Semántica difícil de predecir

No existe un mecanismo óptimo para todos los casos. Cada uno tiene un costo.

JavaScript/TypeScript usan pass-by-value para primitivos y pass-by-sharing para objetos (lo veremos en F-12). El pass-by-name casi no se usa hoy (Scala lo tiene como call-by-name) pero es históricamente importante.

Cita: “The implementation of pass-by-value, -result, -value-result, and -reference, where the run-time stack is used.” [Sebesta 2019, §9.5, p. 389]. Y: “Pass-by-reference is a second implementation model for inout-mode parameters. Rather than copying data values back and forth, however, as in pass-by-value-result, the pass-by-reference method transmits an access path, usually just an address, to the called subprogram.” [Sebesta 2019, §9.5, p. 389]. Gabbrielli: “The mode in which actual parameters are paired with formal parameters, and the semantics which results from this, is called the parameter passing discipline.” [Gabbrielli & Martini 2023, Cap. 7, p. 136]

5.3 Bloque C — Tres lenguajes, tres decisiones de pasaje [F-09 a F-11]

No se enseña cada lenguaje: se **usa cada uno para iluminar una decisión de diseño.**

5.3.1 Go — aislamiento de pass-by-value [F-09]

```
func incrementar(n int) int {  
    n = n + 1  
    return n  
}  
edad := 20  
siguiente := incrementar(edad) // edad sigue siendo 20
```

- La expresión real se evalúa **antes de entrar**.
- Su valor inicializa una **nueva variable local**.
- La asignación afecta solo esa copia.
- El retorno explícito comunica el resultado.

Modificar el formal **no modifica el argumento**. Si querés modificar el argumento en Go, tenés que pasar un **puntero explícitamente** (que también se pasa por valor). El contrato lo hace visible.

 **Pregunta típica:** ¿Go tiene referencias? — No. Go siempre es pass-by-value; para mutar pasás un puntero. Conceptualmente igual a C, donde los punteros son el mecanismo para simular referencia.

Cita: “C uses pass-by-value. Pass-by-reference (inout mode) semantics is achieved by using pointers as parameters. The value of the pointer is made available to the called function and nothing is copied back. However, because what was passed is an access path to the data of the caller, the called function can change the caller’s data.” [Sebesta 2019, §9.5, p. 389]. Gabbrielli: “Like in C, C++, Pascal and Java, when we do not explicitly indicate any parameter-passing mode, it is to be understood that parameter is to be passed by value.” [Gabbrielli & Martini 2023, Cap. 7, p. 136]

5.3.2 Rust — aliasing mutable restringido [F-10]

```
fn incrementar(n: &mut i32) {  
    *n += 1;  
}  
let mut x = 10;  
incrementar(&mut x);  
// sumar(n: &mut i32, m: &mut i32) exige dos referencias exclusivas
```

- Una referencia mutable exige **acceso exclusivo**.
- El formal se vincula con la **ubicación** del argumento real — no hay copia.
- Asignar al formal modifica directamente el dato del llamador.
- Rust permite **muchas referencias inmutables o una sola mutable**.
- El **borrow checker** rechaza aliasing mutable antes de ejecutar.

El contrato no es solo tipos: es **reglas de aliasing**. Dos `&mut` simultáneos al mismo dato serían rechazados en compilación. Esto elimina una clase entera de bugs antes de ejecutar.

Cita: “Pass-by-reference: the formal parameter is bound to the location of the actual parameter. Access to the formal parameters in the called subprogram is by indirect addressing from the stack location of the address.” [Sebesta 2019, §9.5, p. 389]

5.3.3 Swift — mutación explícita con inout [F-11]

```
func avanzar(posicion: inout Int, pasos: Int) -> Bool {
    posicion += pasos
    return posicion >= 100
}
var posicion = 90
let final = avanzar(posicion: &posicion, pasos: 15)
```

- Los parámetros comunes son **constantes** dentro de la función.
- `inout` permite leer y escribir el argumento del llamador.
- La llamada usa `&` para hacer **visible la posible mutación** en el sitio de llamada.
- Swift restringe accesos superpuestos al mismo almacenamiento.

La idea es que el contrato sea visible **en el sitio de llamada**, no solo en la definición. El `&` es documentación visual: quien lee `avanzar(posicion: &posicion, ...)` sabe que `posicion` puede cambiar.

 **Detalle semántico:** `inout` de Swift es **value-result**: se copia al entrar, se copia al salir. Pero el `&` lo hace visible, a diferencia del value-result clásico.

Cita: Sebesta trata pass-by-value-result (que implementa `inout`) en §9.5: *“Pass-by-value-result is an implementation model for inout-mode parameters in which actual values are copied. It is in effect a combination of pass-by-value and pass-by-result.”* [Sebesta 2019, §9.5, p. 389]

5.4 Bloque D — Compartir objetos: copia vs. aliasing [F-12 a F-13]

5.4.1 Pass-by-sharing: separar variable y objeto [F-12]

```
const usuario = { nombre: "Matias", roles: ["lector"] };
function cambiar(u: typeof usuario): void {
    u.roles.push("editor"); // muta el objeto compartido
    u = { nombre: "Otro", roles: [] }; // reasigna solo la variable formal
}
cambiar(usuario);
// usuario.nombre permanece "Matias"
// usuario.roles contiene "lector" y "editor"
```

Pass-by-sharing **separa variable y objeto**:

- `u.roles.push("editor")` **muta el objeto compartido** → sobrevive a la llamada.
- `u = { nombre: "Otro", roles: [] }` **reasigna la variable formal** → no sobrevive.

Después de la llamada, `usuario.nombre` sigue siendo `"Matias"` pero `usuario.roles` tiene `"lector"` y `"editor"`. **Compartir referencias no implica necesariamente compartir cambios.** La mutabilidad define si compartir es peligroso.

 **Trampa frecuente:** ¿Esto es *pass-by-reference*? — **No**. En *pass-by-reference*, reasignar el formal cambia el real. En *pass-by-sharing*, no. La distinción es crucial. JavaScript/TypeScript hacen *pass-by-sharing* con objetos.

Cita: Sebesta trata el caso de Java (object references) en §9.5: el reference se pasa por valor, el objeto se comparte [Sebesta 2019, §9.5, p. 389].

5.4.2 Una matriz grande: copia vs. aliasing [F-13]

Mecanismo	Costo principal	Riesgo principal	Lectura del contrato
Valor	Copiar toda la matriz	Consumo de memoria	Aislamiento total
Referencia mutable	Sin copia inicial	Aliasing y efectos laterales	Mutación compartida
Valor-resultado	Copia al entrar y salir	Orden de copia final	Cambio diferido
Referencia inmutable + resultado	Copia solo del resultado	Construcción de nueva matriz	Flujo explícito

Elegir un mecanismo exige **balancear riesgos**. No hay respuesta universal: depende del tamaño del dato, de la frecuencia de llamada y de cuánto confiás en el llamador. La “referencia inmutable + resultado” es el estilo de funciones puras que retornan un nuevo valor en lugar de mutar.

Cita: “Let us note how this is an expensive mode when the value parameter is bound to a large data structure. In such a case, the entire structure is copied to the formal. On the other hand, the cost of accessing the formal parameter is minimal.” [Gabbrielli & Martini 2023, Cap. 7, p. 136]

5.5 Bloque E — Callbacks: contrato, suspensión y escape [F-14 a F-15]

5.5.1 Un callback es parte del contrato [F-14]

Un **callback** es un subprograma recibido como parámetro y ejecutado por otro subprograma. Pasar comportamiento exige definir **protocolo, efectos y frecuencia**:

- La firma establece entradas y retorno del callback.
- El contrato debe aclarar **cuántas veces y cuándo** será invocado.
- También importa si puede **fallar, suspenderse o retenerse**.
- Una callback retenida puede **extender la vida** de su entorno capturado.

```
type Comparador<T> = (a: T, b: T) => number;
function ordenar<T>(xs: T[], comparador: Comparador<T>): T[] {
  return [...xs].sort(comparador);
}
```

La closure ya la estudiaste (Tema 09.2); aquí importa su **impacto contractual**. El callback es parte del contrato del **llamador**, no del llamado.

Cita: “Although the idea is natural and seemingly simple, the details of how it works can be confusing. If only the transmission of the subprogram code was necessary, it could be done by passing a single pointer. However, two complications arise.” [Sebesta 2019, §9.6 Parameters That Are Subprograms, p. 393]. Sebesta trata además el llamado indirecto en §9.7 [Sebesta 2019, §9.7, p. 389].

5.5.2 Síncrono, suspendible y escapante [F-15]

Kotlin distingue callback síncrono de suspendible:

```
fun <T, R> transformar(xs: List<T>, f: (T) -> R): List<R> {
    // f debe devolver R antes de continuar
    return xs.map { x -> f(x) }
}

suspend fun <T, R> transformarAsync(xs: List<T>, f: suspend (T) -> R): List<R> {
    // f puede pausar su ejecución sin bloquear el hilo
    val resultado = mutableListOf<R>()
    for (x in xs) { resultado.add(f(x)) }
    return resultado
}
```

- `(T) -> R` debe completar antes de devolver el control — es **síncrono**.
- `suspend (T) -> R` puede suspender y reanudarse — el modificador comunica un **efecto** que el tipo de retorno no expresa solo.

Swift marca `@escaping` para hacer visible el escape:

```
var handlers: [(Evento) -> Void] = []
func registrar(_ handler: @escaping (Evento) -> Void) {
    handlers.append(handler) // el callback escapa: sobrevive a la llamada
}
func emitir(_ evento: Evento) {
    for h in handlers { h(evento) }
}
```

- Un callback **no diferido** se ejecuta durante la llamada.
- Un callback **diferido** (escapante) se almacena y ejecuta más tarde.
- Retener callbacks puede crear **ciclos de referencias** y recursos vivos.

 **Ortogonalidad:** sincronía y escape son **dimensiones ortogonales**. Un callback suspendible puede escapar, y uno síncrono puede no escapar. Son cuatro combinaciones, no dos.

El escape cambia **duración, ownership y manejo de errores**. Por eso Swift lo hace visible en el tipo con `@escaping`.

5.6 Bloque F — Variación: sobrecarga, dispatch y trait [F-16 a F-18]

5.6.1 Herramientas para expresar variación [F-16]

Herramienta	Variación modelada	Ejemplo apropiado	Riesgo de mal uso
Sobrecarga	Protocolos estáticos distintos	<code>parsear(string)</code> y <code>parsear(bytes)</code>	Ambigüedad
Unión sellada	Conjunto cerrado de casos	Estado de una operación	Acoplar todos los casos
Genérico/trait	Capacidad uniforme	Algoritmo sobre ordenables	Restricción excesiva
Interfaz dinámica	Implementaciones abiertas	Plugins	Fallas tardías de integración

Y la pregunta clave: **¿cuándo se selecciona la implementación?**

Mecanismo	Cuándo se selecciona	Información usada	Costo principal
Sobrecarga	Compilación	Tipos y argumentos	Complejidad de resolución
Despacho virtual	Ejecución	Tipo dinámico del receptor	Indirección
Callback	Ejecución	Valor función recibido	Indirección y captura
Trait/genérico	Compilación o ejecución	Estrategia del lenguaje	Código generado o tabla dinámica

Un nombre puede resolverse en momentos diferentes. Resolución estática, despacho dinámico e indirección **no son equivalentes**.

Cita: “A polymorphic subprogram takes parameters of different types on different activations. Overloaded subprograms provide a particular kind of polymorphism called *ad hoc* polymorphism. Overloaded subprograms need not behave similarly.” [Sebesta 2019, §9.9, p. 389]. Y: “Parametric polymorphism is provided by a subprogram that takes generic parameters that are used in type expressions.” [Sebesta 2019, §9.10, p. 389]

5.6.2 Kotlin — sobrecarga entre cuerpos distintos [F-17]

```
fun area(radio: Double): Double = Math.PI * radio * radio
fun area(base: Double, altura: Double): Double = base * altura

area(3.0)           // círculo
area(3.0, 4.0)      // rectángulo
```

- Cada sobrecarga tiene su **propio cuerpo**.
- El compilador busca la **mejor coincidencia** según los argumentos.
- Conversiones implícitas y parámetros por defecto pueden crear **ambigüedad**.
- Si el algoritmo es uniforme, una **plantilla** (genérico) evita duplicación.

La sobrecarga es **ad hoc**: cada cuerpo puede hacer algo distinto. TypeScript tiene overload signatures, pero **una sola implementación** — es distinto a Kotlin.

Cita: “An overloaded subprogram is one that has the same name as another subprogram in the same referencing environment.” [Sebesta 2019, §9.9, p. 389]. Y: “Overloaded subprograms that have default parameters can lead to ambiguous subprogram calls.” [Sebesta 2019, §9.9, p. 389]

5.6.3 Rust — impl Trait vs. dyn Trait [F-18]

Un **trait** en Rust es una abstracción de comportamiento: define **qué** operaciones debe soportar un tipo, sin decir **cómo**.

```
trait Tarea {
    fn ejecutar(&self);
}
struct Email;
impl Tarea for Email {
    fn ejecutar(&self) { println!("Enviando email"); }
}

fn ejecutar_estatico(t: &impl Tarea) { t.ejecutar(); } // dispatch estático
fn ejecutar_dinamico(t: &dyn Tarea) { t.ejecutar(); } // dispatch dinámico
```

- **impl Trait** permite especialización estática y optimización — el compilador genera código específico.
- **dyn Trait** acepta implementaciones heterogéneas mediante indirección — el compilador solo sabe que cumple el Trait, no cuál es.
- Ambos expresan capacidades, pero producen **representaciones distintas**.
- La elección afecta **tamaño de código, rendimiento y flexibilidad**.

 **Cuándo usar dyn** : cuando tenés una colección heterogénea de tipos que comparten un trait y no conocés los tipos en compilación. **impl Trait** es conceptualmente un genérico con restricción de trait.

Cita: “A generic subprogram is one whose computation can be done on data of different types in different activations.” [Sebesta 2019, §9.10, p. 389]. Gabbrielli sobre subtipos: “By virtue of subtype compatibility, foo can receive as an argument a value of any subclass of A.” [Gabbrielli & Martini 2023, Cap. 7, p. 295]

5.7 Bloque G — Abstracción genérica y sus costos [F-19]

La abstracción genérica tiene **costos de implementación**. Tres estrategias:

Estrategia	Idea	Ventaja	Costo
Especialización por tipo	Generar código por instanciación	Optimización específica	Mayor binario
Implementación compartida	Compartir implementación runtime	Menor duplicación	Menor información runtime
Preservación selectiva	Conservar ciertos tipos	Inspección selectiva	Reglas más complejas

TypeScript usa **type erasure**: en runtime no existe `T`.

```
// genérico: una sola definición sirve para distintos tipos
function identidad<T>(x: T): T { return x; }

const a = identidad<number>(10);
const b = identidad<string>("hola");
// en runtime no existe T — los tipos genéricos se borran

// preservación selectiva: pasar información explícitamente
function esTipo<T>(x: unknown, check: (v: unknown) => v is T): boolean {
  return check(x);
}
const esNumero = esTipo<number>(10, (v): v is number => typeof v === "number");
```

Si necesitas información en runtime, debes **pasarla explícitamente** (como el `check` en `esTipo`). Especializar o compartir código **intercambia rendimiento, tamaño y flexibilidad**.

 **Contraste entre lenguajes:** Rust y C++ **especializan** (monomorfización: generan código por cada tipo). Java y TypeScript **comparten** (type erasure: una sola implementación). La decisión afecta el tamaño del binario y la información disponible en runtime.

Cita: “Parametric polymorphism is provided by a subprogram that takes generic parameters that are used in type expressions that describe the types of the parameters of the subprogram. Different instantiations of such subprograms can be given different generic parameters.” [Sebesta 2019, §9.10, p. 389]. Louden sobre polimorfismo paramétrico: “This type of polymorphism is called parametric polymorphism because a is essentially a type parameter that can be replaced by any type expression.” [Louden & Lambert 2012, Cap. 8, p. 372]

5.8 Bloque H — Ejecución: activation records, async y enlaces [F-20 a F-23]

5.8.1 El activation record materializa una llamada [F-20]

activation record (AR) = memoria de una activación. **activación** = instancia concreta de ejecución. Es la estructura de memoria que conserva el estado de una activación mientras la llamada está en ejecución.

Componente	Función durante la llamada
Parámetros	Comunican datos desde el llamador
Variables locales	Conservan el estado privado de esa activación
Dirección de retorno	Indica dónde continuar al terminar
Dynamic link	Apunta al AR del llamador — <i>¿quién me llamó?</i>
Valor de retorno	Comunica el resultado
Static link, si aplica	Permite acceder a variables no locales léxicas — <i>¿dónde busco variables no locales?</i>

El AR se crea al llamar y se libera al retornar. Típicamente vive en el **stack**; las closures que escapan pueden ir al **heap**.

Cita: Sebesta trata el activation record instance (ARI) y sus componentes en §10.2 Implementing “Simple” Subprograms [Sebesta 2019, §10.2, p. 441]. Gabbrielli: *“Dynamic chain pointer. This field stores a pointer to the previous activation record on the stack. Some authors call this pointer the dynamic link or control link.”* [Gabbrielli & Martini 2023, Cap. 7, p. 106]

5.8.2 Call y return: administrar el stack [F-21]

Llamar crea un nuevo contexto; retornar restaura el anterior. Seis momentos:

Momento	Acción
1. Preparar llamada	Evaluar argumentos y establecer parámetros
2. Crear activación	Reservar activation record
3. Transferir control	Guardar retorno y saltar al punto de entrada
4. Ejecutar	Usar parámetros, locales y enlaces
5. Retornar	Producir resultado y restaurar al llamador
6. Liberar	Retirar el activation record terminado

Ejemplo del `clase_dada.txt` :

```
fun sumar(a: Int, b: Int): Int {
    val z = a + b
    return z
}
sumar(2, 3)
```

Al llamar `sumar(2, 3)` se genera:


```
Activation Record(sumar)
-----
a = 2
b = 3
z = ?
ret addr -> main
dynamic link -> AR(main)
```

Cuando `sumar` retorna, el AR se libera y el control vuelve a `main`. En recursión, **cada llamada recursiva crea un AR distinto** — por eso la recursión consume stack.

Cita: “The required actions of a subprogram return are less complicated than those of a call. If the subprogram has parameters that are out mode or inout mode and are implemented by copy, the first action of the return process is to move the local values of the associated formal parameters to the actual parameters.” [Sebesta 2019, §10.1 General Semantics of Calls and Returns, p. 441]. Y: “Activating a subprogram requires the dynamic creation of an instance of the activation record for the subprogram.” [Sebesta 2019, §10.2, p. 441]

5.8.3 async extiende el modelo de ejecución [F-22]

```
async function cargar(id: string): Promise<Usuario> {
  const respuesta = await fetch(`/usuarios/${id}`);
  return respuesta.json() as Promise<Usuario>;
}
```

- Antes del primer `await`, la función ejecuta como una **llamada ordinaria**.
- Al suspenderse, debe conservar **parámetros, locales y punto de continuación**.
- El compilador/runtime materializa una **máquina de estados reanudable**.
- Los stack traces async reconstruyen una **cadena lógica**, no siempre el stack físico original.

`async` es una **extensión** del modelo de ejecución, no un modelo distinto. La suspensión conserva estado **sin mantener el stack síncrono completo**. La máquina de estados típicamente vive en el **heap** porque debe sobrevivir a la suspensión.

 **Aclaración frecuente:** ¿*async* crea un *thread*? — **No**. *async* es concurrencia de **espera**, no paralelismo. El hilo se libera al suspender.

Cita: Sebesta trata la semántica general de calls y returns y la transferencia de control en §10.1 [Sebesta 2019, §10.1, p. 441]. Sobre subprogramas anidados y acceso a variables no locales: “If the language supports nested subprograms, the call process must create some mechanism to provide access to nonlocal variables.” [Sebesta 2019, §10.4 Nested Subprograms, p. 441]

5.8.4 Dynamic link vs. static link [F-23]

Dynamic link y **static link** responden preguntas diferentes:

- **Dynamic link** apunta al AR del llamador. Permite **restaurar el stack al retornar**. Responde: *¿quién me llamó?* (runtime, orden de llamadas).
- **Static link** apunta hacia un **ancestro léxico**. Permite **buscar variables no locales** con alcance estático. Responde: *¿dónde busco variables no locales?* (léxico, estructura del programa).

El orden de llamadas y la estructura del programa no siempre coinciden. El dynamic link sigue el orden de llamadas en runtime; el static link sigue la estructura léxica del código fuente. El static link solo aplica en lenguajes con **subprogramas anidados** y alcance estático.

 **Pregunta típica:** *¿JavaScript necesita static link?* — No. JavaScript no tiene subprogramas anidados con alcance estático en el sentido clásico; usa closures y environment chains.

Cita: *“Static and dynamic links must be maintained in the activation record instances. The static link is to allow references to nonlocal variables in static-scoped languages.”* [Sebesta 2019, §10.4, p. 441]. Y: *“Dynamic chain pointer. This field stores a pointer to the previous activation record on the stack. Some authors call this pointer the dynamic link or control link.”* [Gabbrielli & Martini 2023, Cap. 7, p. 106]

5.9 Bloque I — Cierre: las decisiones de diseño están conectadas [F-24]

Seis preguntas sintetizan la clase. Cada una es una decisión, y cada decisión tiene consecuencias observables:

Pregunta	Decisión	Consecuencia
¿Qué acepta y retorna?	Perfil y protocolo	Verificación de llamadas
¿Cómo circulan datos?	Modos y mecanismos de pasaje	Copias, aliasing y efectos
¿Qué efectos produce?	Mutación, falla, suspensión	Obligaciones del llamador
¿Puede retener callbacks?	Escapante o no escapante	Duración y recursos
¿Cómo selecciona implementación?	Sobrecarga, trait o dispatch	Costo y extensibilidad
¿Cómo se ejecuta?	Frames, continuaciones y ABI	Call, return y depuración

El hilo conductor: del contrato visible al mecanismo de ejecución. El contrato que escribís determina el mecanismo que se ejecuta.

Cita (síntesis): Sebesta recorre el camino completo del contrato del subprograma (§9.2) a la implementación con activation records (§10.2) y nested subprograms (§10.4) [Sebesta 2019, Cap. 9–10]. Gabbrielli sintetiza del procedimiento como unidad de modularización (p. 136) a la implementación con dynamic chain pointer (p. 106) [Gabbrielli & Martini 2023, Cap. 7].

6. Ejemplos trabajados

Ejemplo 1 — Trazar una llamada: pass-by-value (Go) vs. pass-by-reference restringida (Rust)

Consigna: predecí el valor de `edad` y `x` después de cada llamada, y explicá por qué.

Go (pass-by-value):

```
func incrementar(n int) int {  
    n = n + 1  
    return n  
}  
edad := 20  
siguiente := incrementar(edad)  
// ¿edad? ¿siguiente?
```

Rust (pass-by-reference con &mut):

```
fn incrementar(n: &mut i32) {  
    *n += 1;  
}  
let mut x = 10;  
incrementar(&mut x);  
// ¿x?
```

Solución paso a paso:

Go:

1. Se evalúa `edad` → `20`.
2. Se crea una **nueva variable local** `n` con valor `20` (copia).
3. `n = n + 1` → `n` vale `21` (la copia local).
4. Se retorna `21` → `siguiente = 21`.
5. El AR de `incrementar` se libera. `n` desaparece.
6. `edad` **sigue siendo 20** — el formal era una copia, modificarlo no toca el argumento.

Rust:

1. Se pasa la **ubicación** de `x` (no una copia) al parámetro `n: &mut i32`.
2. `*n += 1` desreferencia y escribe **directamente** en la ubicación de `x`.
3. No hay copia; el AR de `incrementar` guarda la referencia, no el valor.
4. `x` **vale 11** — la mutación sobrevive porque el formal era un acceso a la ubicación del real.

Conclusión: Go aísla copiando (modo `in`); Rust muta con acceso exclusivo (modo `inout` con restricción de aliasing). El borrow checker de Rust rechazaría `sumar(&mut x, &mut x)` porque dos `&mut` al mismo dato violan la regla de exclusividad.

 **Por qué importa:** la diferencia no es sintáctica — es **contractual**. En Go el llamador sabe que `edad` no cambia mirando la firma (`n int`). En Rust el llamador sabe que `x` puede cambiar mirando `&mut i32` . El contrato hace visible la mutación.

Ejemplo 2 — Analizar el activation record de un subprograma con llamada anidada

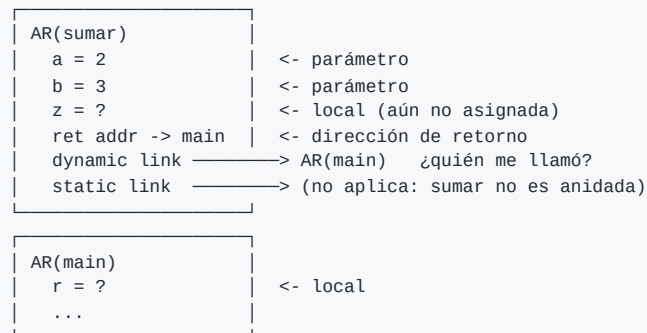
Consigna: dado el siguiente código, dibujá los activation records en el stack cuando la ejecución llega a `z = a + b` dentro de `sumar` , e identificá dynamic link y static link.

```
fun main() {  
    val r = sumar(2, 3)  
}  
  
fun sumar(a: Int, b: Int): Int {  
    val z = a + b    // <- la ejecución está acá  
    return z  
}
```

Solución paso a paso:

1. `main` empieza: se crea `AR(main)` en el stack.
2. `main` llama a `sumar(2, 3)` : se evalúan los argumentos (`2` , `3`).
3. Se crea `AR(sumar)` encima de `AR(main)` :

Stack (tope arriba):



4. Se ejecuta `z = a + b` → `z = 5` .
5. `return z` → el valor `5` se coloca en el lugar de retorno.
6. Se libera `AR(sumar)` ; el control vuelve a `main` en la dirección de retorno.
7. `r = 5` .

Puntos clave:

- **Dynamic link** de `AR(sumar)` apunta a `AR(main)` — responde *¿quién me llamó?* y permite restaurar el stack al retornar.
- **Static link** no aplica acá porque `sumar` **no es anidada** en `main` léxicamente (en Kotlin/TypeScript las funciones top-level no tienen padre léxico). Aparece solo en lenguajes con subprogramas anidados y alcance estático.
- Si `sumar` fuera recursiva (`sumar(2, sumar(1, 1))`), habría **dos AR de `sumar`** distintos en el stack, cada uno con su propio `dynamic link`.

 **Trampa:** el AR guarda el estado de una **activación**, no del código. El código de `sumar` es uno solo; los AR son muchos (uno por llamada).

Ejemplo 3 — Distinguir sobrecarga vs. genérico vs. dispatch (estático vs. dinámico)

Consigna: para cada fragmento, decidí (a) qué herramienta de variación usa, (b) en qué momento se selecciona la implementación (compilación o ejecución), y © qué costo principal tiene.

Fragmento A — Kotlin:

```
fun area(radio: Double): Double = Math.PI * radio * radio
fun area(base: Double, altura: Double): Double = base * altura
area(3.0, 4.0)
```

Fragmento B — TypeScript:

```
function identidad<T>(x: T): T { return x; }
identidad<number>(10);
```

Fragmento C — Rust:

```
fn ejecutar_dinamico(t: &dyn Tarea) { t.ejecutar(); }
```

Solución:

	Fragmento A (Kotlin)	Fragmento B (TypeScript)	Fragmento C (Rust)
(a) Herramienta	Sobrecarga (ad hoc)	Genérico (paramétrico)	Dispatch dinámico (<code>dyn Trait</code>)
(b) Momento de selección	Compilación	Compilación (type erasure en runtime)	Ejecución
© Costo principal	Complejidad de resolución (ambigüedad posible)	Implementación compartida: menor info en runtime	Indirección (tabla virtual)
Cuerpos distintos	Sí — cada sobrecarga tiene su cuerpo	No — una sola implementación	Depende del tipo receptor en runtime
Riesgo	Ambigüedad con defaults/conversiones	No hay info de <code>T</code> en runtime	Fallas tardías si el tipo no implementa el trait

Conclusión:

- **Sobrecarga (A)** resuelve entre **cuerpos distintos** en compilación, usando tipos y argumentos. Es ad hoc: cada cuerpo puede hacer algo distinto.
- **Genérico (B)** usa **una sola implementación** para distintos tipos. TypeScript borra `T` en runtime (type erasure), así que no hay información de tipo en runtime salvo que la pases explícita.
- **`dyn Trait` ©** despacha en **ejecución** según el tipo dinámico del receptor, con indirección (tabla virtual). A diferencia de `impl Trait` (estático, especialización), `dyn` acepta tipos heterogéneos pero paga indirección.

 **Regla mnemotécnica:** *sobrecarga = muchos cuerpos, compilación; genérico = un cuerpo, compilación (o runtime según estrategia); dyn = muchos tipos, ejecución.*

Ejemplo 4 — Pass-by-sharing: mutación vs. reasignación

Consigna: predecí el estado de `usuario` después de la llamada y explicá la diferencia entre mutar y reasignar.

```
const usuario = { nombre: "Matias", roles: ["lector"] };
function cambiar(u: typeof usuario): void {
  u.roles.push("editor");
  u = { nombre: "Otro", roles: [] };
}
cambiar(usuario);
```

Solución paso a paso:

1. `usuario` es una variable que **referencia** un objeto `{ nombre: "Matias", roles: ["lector"] }`.
2. Al llamar `cambiar(usuario)`, el parámetro formal `u` recibe **una copia de la referencia** (no del objeto). `u` y `usuario` apuntan al **mismo objeto**.
3. `u.roles.push("editor")` muta el **objeto compartido**. Como `u` y `usuario` apuntan al mismo objeto, el cambio es visible desde afuera. Ahora el objeto tiene `roles: ["lector", "editor"]`.
4. `u = { nombre: "Otro", roles: [] }` **reasigna la variable formal** `u` para que apunte a un **nuevo objeto**. Esto **no afecta** a `usuario`, que sigue apuntando al objeto original.
5. Después de la llamada:
 - `usuario.nombre === "Matias"` (el objeto original no fue reasignado).
 - `usuario.roles === ["lector", "editor"]` (la mutación del objeto compartido sobrevivió).

Conclusión: pass-by-sharing **separa variable y objeto**. Mutar el objeto compartido sobrevive; reasignar la variable formal no. Esta es la diferencia crucial con pass-by-reference, donde reasignar el formal **sí** cambiaría el real.

 **Por qué cuesta:** la trampa es pensar “los objetos se pasan por referencia”. En realidad, **la referencia se pasa por valor** — dos variables distintas apuntan al mismo objeto. Por eso mutar sí y reasignar no.

7. Puntos clave (cheat-sheet)

 **Cómo usar esta sección:** es el resumen para repasar antes del examen. Si podés explicar cada línea sin mirar la guía, estás listo.

El contrato

- **Subprograma:** abstracción de acción; se razona por **contrato**, no por instrucciones. Punto de entrada único, retorno de control, cuerpo oculto.
- **Código ≠ activación:** el código es uno; cada llamada crea una activación nueva.
- **Definición vs. llamada:** roles distintos que forman un contrato. Parámetros **formales** (definición, variables locales del llamado) vs. **reales** (llamada, valores aportados).
- **Perfil:** número, orden y tipos de parámetros (sin retorno).
- **Protocolo:** perfil + tipo de retorno. Reaparece en sobrecarga y orden superior.
- **Procedimiento:** efecto, resultado implícito, secuencia. **Función:** cálculo, resultado explícito, expresión.

Parámetros

- **Modos (dirección del flujo):** `in` (recibe), `out` (produce), `inout` (ambas). **Primero el modo, después el mecanismo.**
- **Permiso mínimo:** consultar (`readonly`), modificar (`&mut` / `inout`), consumir (`ownership`), producir (retorno).
- **Efectos observables:** retorno, mutación, falla, suspensión, cancelación. La firma tipada no captura todo.
- **Mecanismos:** valor (`in` , aísla), resultado (`out`), valor-resultado (`inout` , sin aliasing), referencia (`inout` , aliasing), nombre (`inout` , reevalúa).

Lenguajes reales

- **Go:** pass-by-value; para mutar, pasás un puntero. Aislamiento total.
- **Rust:** `&mut` = acceso exclusivo; borrow checker rechaza aliasing mutable.
- **Swift:** `inout` = value-result visible; `&` en la llamada hace visible la mutación.
- **Pass-by-sharing:** mutar el objeto compartido sobrevive; reasignar la variable formal no. **No es pass-by-reference.**

Callbacks

- **Callback:** subprograma como parámetro. Contrato debe aclarar **frecuencia, fallo, retención**.
- **Síncrono** (`(T) -> R`) vs. **suspendible** (`suspend (T) -> R`): el modificador comunica un efecto.
- **No escapante** (se ejecuta durante la llamada) vs. **escapante** (`@escaping` , se ejecuta después): el escape cambia duración y ownership.
- Sincronía y escape son **ortogonales**.

Variación

- **Sobrecarga:** protocolos estáticos distintos, cuerpos distintos, selección en compilación. Riesgo: ambigüedad.
- **Unión sellada:** conjunto cerrado de casos. Riesgo: acoplar casos.
- **Genérico/trait:** capacidad uniforme. Riesgo: restricción excesiva.
- **Interfaz dinámica:** implementaciones abiertas. Riesgo: fallas tardías.
- `impl Trait` (estático, especialización, mayor binario) vs. `dyn Trait` (dinámico, indirección, heterogéneo).

Genéricos

- **Especialización por tipo:** código por instanciación (Rust, C++). Mayor binario, mayor optimización.
- **Implementación compartida:** una sola implementación (Java, TypeScript con type erasure). Menor info en runtime.
- **Preservación selectiva:** conservar ciertos tipos pasando información explícita.

Ejecución

- **Activation record:** parámetros, locales, dirección de retorno, dynamic link, valor de retorno, static link (si aplica).
- **Calling sequence:** preparar → crear → transferir → ejecutar → retornar → liberar.
- **Recursión:** cada llamada crea un AR distinto → consume stack.
- **async:** máquina de estados reanudable; conserva estado sin mantener el stack síncrono. No crea thread.
- **Dynamic link:** *¿quién me llamó?* (runtime, orden de llamadas).
- **Static link:** *¿dónde busco variables no locales?* (léxico, estructura del programa). Solo en subprogramas anidados.

Hilo conductor

Del contrato visible al mecanismo de ejecución. Seis preguntas, seis decisiones conectadas: qué acepta/retorna → cómo circulan datos → qué efectos produce → puede retener callbacks → cómo selecciona implementación → cómo se ejecuta.

8. Autoevaluación

 **Cómo usarla:** intentá responder **sin mirar la guía**. Después abrí cada `<details>` para verificar. Las preguntas están ordenadas por nivel Bloom (de Recordar a Crear). Si trabás en una de Aplicar/Analizar, volvé a la sección correspondiente del desarrollo teórico.

#	Nivel Bloom	Habilidad
1	Recordar	Reconocer definiciones
2	Recordar	Enumerar
3	Entender	Explicar
4	Entender	Comparar
5	Aplicar	Predecir salida
6	Aplicar	Trazar ejecución
7	Analizar	Distinguir conceptos
8	Analizar	Clasificar
9	Evaluar	Justificar elección
10	Crear	Diseñar contrato

Pregunta 1 (Recordar) — Definición de perfil y protocolo

¿Qué incluye el **perfil** de un subprograma y qué agrega el **protocolo**? ¿Por qué la distinción importa en sobrecarga?

► Respuesta

Pregunta 2 (Recordar) — Componentes del activation record

Enumera los **seis** componentes del activation record y, para cada uno, una frase que diga su función.

► Respuesta

Pregunta 3 (Entender) — Modo vs. mecanismo

Explicá con tus palabras por qué **modo in** y **pass-by-value** no son lo mismo. Dad un ejemplo de un mecanismo distinto que también implemente **in**.

► Respuesta

Pregunta 4 (Entender) — Procedimiento vs. función

Compará procedimiento y función según **intención, resultado y composición**. ¿Por qué Kotlin usa `Unit` y Rust exige declarar el tipo de retorno cuando no es `()`?

► Respuesta

Pregunta 5 (Aplicar) — Predecir salida: pass-by-sharing

¿Cuál es el estado de `usuario` después de ejecutar este código TypeScript? Explicá paso a paso.

```
const usuario = { nombre: "Matias", roles: ["lector"] };
function cambiar(u: typeof usuario): void {
  u.roles.push("editor");
  u = { nombre: "Otro", roles: [] };
}
cambiar(usuario);
// ¿usuario.nombre? ¿usuario.roles?
```

► Respuesta

Pregunta 6 (Aplicar) — Trazar el activation record de `sumar(2, 3)`

Dibujá el activation record de `sumar` cuando la ejecución llega a `val z = a + b`, e indicá qué apunta el dynamic link y (si aplica) el static link.

```
fun sumar(a: Int, b: Int): Int {
  val z = a + b
  return z
}
sumar(2, 3)
```

► Respuesta

Pregunta 7 (Analizar) — Sobrecarga vs. genérico vs. dispatch

Para cada fragmento, decidí (a) qué herramienta de variación usa, (b) cuándo se selecciona la implementación, y © un riesgo específico.

```
// A
fun area(radio: Double): Double = Math.PI * radio * radio
fun area(base: Double, altura: Double): Double = base * altura
```

```
// B
function identidad<T>(x: T): T { return x; }
```

```
// C
fn ejecutar(t: &dyn Tarea) { t.ejecutar(); }
```

► Respuesta

Pregunta 8 (Analizar) — Clasificar mecanismos por modo

Para cada mecanismo, indicá qué modo implementa y un escenario donde su riesgo principal se manifiesta.

Mecanismo	Modo	Escenario de riesgo
Valor	?	?
Resultado	?	?
Valor-resultado	?	?
Referencia	?	?
Nombre	?	?

► Respuesta

Pregunta 9 (Evaluar) — Elegir mecanismo para una matriz grande

Tenés que implementar una función `normalizar(matriz: number[][]): number[][]` que recibe una matriz grande (digamos, 1000×1000), la procesa y produce una nueva matriz normalizada. Justificá qué combinación de **modo + mecanismo** usarías y por qué, comparando al menos dos alternativas.

► Respuesta (ejemplo de justificación)

Pregunta 10 (Crear) — Diseñar un contrato de callback

Diseñá la firma TypeScript de una función `procesarLotes` que recibe un arreglo de items, un callback que procesa cada item (y puede fallar), y decide: (a) si el callback es síncrono o asíncrono, (b) si es escapante o no, © qué efectos debe declarar el contrato. Justificá cada decisión y escribí la firma con tipos.

► Respuesta (ejemplo de diseño)

9. Glosario



Cómo estudiar el glosario: tapé la definición con la mano y tratá de decir la definición con tus palabras. Si coincidís en lo esencial, está.

Término	Definición
Subprograma	Abstracción de acción con punto de entrada único y retorno de control. Se razona por contrato, no por instrucciones. Unidad fundamental de modularización procedural.
Procedimiento	Subprograma cuya intención es cambiar el estado o producir un efecto. Resultado implícito. Se encadena por secuencia.
Función	Subprograma cuya intención es calcular un valor. Resultado explícito mediante retorno. Se compone dentro de expresiones.
Contrato	Acuerdo entre la definición y la llamada de un subprograma: qué acepta, qué retorna, qué efectos produce, qué permisos exige. El cliente razona con el contrato, no con el cuerpo.
Perfil	Número, orden y tipos de los parámetros de un subprograma. No incluye el tipo de retorno.
Protocolo	Perfil + tipo de retorno. Determina el tipo de la expresión resultante en la llamada. Debe ser único en sobrecarga.
Parámetro formal	Variable local del subprograma llamado, declarada en el encabezado. Se inicializa con el valor del parámetro real al llamar.
Parámetro real (argumento)	Valor o expresión aportada por el llamador en la llamada. Debe ser compatible con el perfil.
Modo in	Dirección del flujo: el llamado recibe información del llamador. Solo entrada.
Modo out	Dirección del flujo: el llamado produce información para el llamador. Solo salida.
Modo inout	Dirección del flujo: la información circula en ambas direcciones. Entrada y salida.
Pass-by-value (valor)	Mecanismo que implementa in : copia el valor del real al formal. Aísla al llamador. Costo: copiar objetos grandes.
Pass-by-result (resultado)	Mecanismo que implementa out : el formal actúa como local y su valor se copia al real al retornar.
Pass-by-value-result (valor-resultado)	Mecanismo que implementa inout : copia al entrar y copia al salir. Evita aliasing durante la llamada. Costo: orden de copia final.
Pass-by-reference (referencia)	Mecanismo que implementa inout : el formal se vincula con la ubicación del real. Evita copias grandes. Riesgo: aliasing y efectos laterales.
Pass-by-name (nombre)	Mecanismo que implementa inout : reevalúa la expresión del real en cada uso del formal. Semántica difícil de predecir. Casi no se usa hoy.
Pass-by-sharing (compartido)	Mecanismo donde se pasa una referencia al objeto por valor. Mutar el objeto compartido sobrevive; reasignar la variable formal no. No es pass-by-reference . Típico de JavaScript/TypeScript/Java con objetos.

Término	Definición
Callback	Subprograma recibido como parámetro y ejecutado por otro subprograma. El contrato debe aclarar frecuencia, fallo y retención.
Callback síncrono	Callback que debe completar antes de devolver el control. En Kotlin: <code>(T) -> R</code> .
Callback suspendible	Callback que puede suspender y reanudarse sin bloquear el hilo. En Kotlin: <code>suspend (T) -> R</code> . El modificador comunica un efecto.
Callback escapante	Callback que se almacena y ejecuta después de la llamada que lo recibió. Swift lo marca con <code>@escaping</code> . Cambia duración, ownership y manejo de errores.
Sobrecarga	Herramienta de variación: mismo nombre, protocolos distintos, cuerpos distintos. Selección en compilación por tipos y argumentos. Polimorfismo ad hoc.
Dispatch (despacho)	Selección de la implementación a ejecutar. Estático : en compilación (sobrecarga, <code>impl Trait</code>). Dinámico : en ejecución, según el tipo dinámico del receptor (<code>dyn Trait</code> , despacho virtual).
Genérico	Subprograma cuya computación puede hacerse sobre datos de distintos tipos en distintas activaciones. Polimorfismo paramétrico. Una sola definición sirve para múltiples tipos.
Trait	Abstracción de comportamiento (Rust): define qué operaciones debe soportar un tipo, sin decir cómo. Similar a una interfaz pero con poder adicional (implementación por defecto, restricciones).
<code>impl Trait</code>	Dispatch estático en Rust: el compilador genera código específico para cada tipo. Especialización, mayor binario.
<code>dyn Trait</code>	Dispatch dinámico en Rust: indirección mediante tabla virtual. Acepta tipos heterogéneos, paga indirección.
Activation record (AR)	Estructura de memoria que conserva el estado de una activación mientras la llamada está en ejecución. Componentes: parámetros, locales, dirección de retorno, dynamic link, valor de retorno, static link (si aplica).
Activación	Instancia concreta de ejecución de un subprograma. El código es uno; las activaciones son muchas (una por llamada).
Calling sequence (secuencia de llamada)	Seis momentos: preparar → crear → transferir → ejecutar → retornar → liberar.
Dynamic link	Campo del AR que apunta al AR del llamador. Responde <i>¿quién me llamó?</i> (runtime, orden de llamadas). Permite restaurar el stack al retornar.
Static link	Campo del AR que apunta hacia un ancestro léxico. Responde <i>¿dónde busco variables no locales?</i> (léxico, estructura del programa). Solo en lenguajes con subprogramas anidados y alcance estático.

Término	Definición
<code>async</code>	Extensión del modelo de ejecución: una suspensión conserva estado sin mantener el stack síncrono completo. El compilador materializa una máquina de estados reanudable. No crea un thread.
Type erasure	Estrategia donde los tipos genéricos se borran al compilar. En runtime no existe <code>T</code> . TypeScript y Java la usan. Contraste: monomorfización (Rust, C++).
Permiso mínimo	Principio: el contrato debe dar al subprograma el menor permiso necesario para su intención. Consultar → readonly; modificar → <code>&mut</code> ; consumir → ownership; producir → retorno.
Efecto observable	Consecuencia de la ejecución visible para el llamador: retorno, mutación, falla, suspensión, cancelación. El contrato moderno los declara explícitamente.

10. Referencias y lecturas recomendadas

Bibliografía principal (verificada en ChromaDB)

1. **Robert W. Sebesta**, *Concepts of Programming Languages*, 11th ed., Pearson, 2019.
 - **Cap. 9: Subprograms** — §9.2 Fundamentals of Subprograms (p. 389); §9.5 Parameter-Passing Methods (p. 389, pass-by-value, -result, -value-result, -reference, modos in/out/inout); §9.6 Parameters That Are Subprograms (p. 393, callbacks); §9.7 Calling Subprograms Indirectly (p. 389); §9.8 Design Issues for Functions (p. 389, side effects); §9.9 Overloaded Subprograms (p. 389, ad hoc polymorphism, protocolo único); §9.10 Generic Subprograms (p. 389, parametric polymorphism).
 - **Cap. 10: Implementing Subprograms** — §10.1 General Semantics of Calls and Returns (p. 441); §10.2 Implementing “Simple” Subprograms (p. 441, activation record instance, componentes); §10.4 Nested Subprograms (p. 441, static link, dynamic link, acceso a variables no locales).
2. **Maurizio Gabbrielli & Simone Martini**, *Programming Languages: Principles and Paradigms*, 2nd ed., Springer, 2023.
 - **Cap. 7** (pp. 106–135): procedimiento vs. función, activation record, dynamic chain pointer / dynamic link / control link.
 - **Cap. 7** (pp. 136–282): parameter passing modes (by value, by reference, read-only), parameter passing discipline, costo de modos, read-only parameter.
 - **Cap. 7** (p. 295): subtype compatibility, polimorfismo.
3. **Kenneth C. Louden & Kenneth A. Lambert**, *Programming Languages: Principles and Practices*, Course Technology, 2012.
 - **Cap. 8** (p. 372): parametric polymorphism, type parameters. Apoyo terminológico sobre subprogramas y polimorfismo.

Fuentes de lenguaje (contrastes)

- **Go Documentation:** function parameters, pass-by-value semantics.
- **Rust Documentation:** ownership, borrowing, `impl Trait` vs. `dyn Trait`, borrow checker.
- **Swift Language Guide:** `inout` parameters, `@escaping` closures.
- **Kotlin Documentation:** function types, `suspend` modifier, overloading.

Lecturas recomendadas (opcionales, para profundizar)

- Sebesta 2019, §10.3 (Implementing Subprograms with Stack-Dynamic Local Variables) y §10.5 (Blocks) — para profundizar en la implementación del stack.
- Sebesta 2019, §10.6 (Implementing Dynamic Scoping) — contraste con el static link.
- Gabbrielli & Martini 2023, §8.7–8.8 — polimorfismo universal vs. ad hoc, para profundizar sobrecarga y genéricos.

 **Cierre de Sofía:** si llegaste hasta acá y podés responder las 10 preguntas de autoevaluación sin mirar, estás listo para el examen de esta clase. Recordá el hilo conductor: **del contrato visible al mecanismo de ejecución**. Cada decisión del contrato (perfil, modos, efectos, callbacks, variación) tiene una contraparte concreta en la ejecución (activation record, call/return, async, enlaces). No son temas sueltos — son el mismo tema visto desde dos lados.

Si un alumno puede estudiarlo solo, lo hicimos bien.
